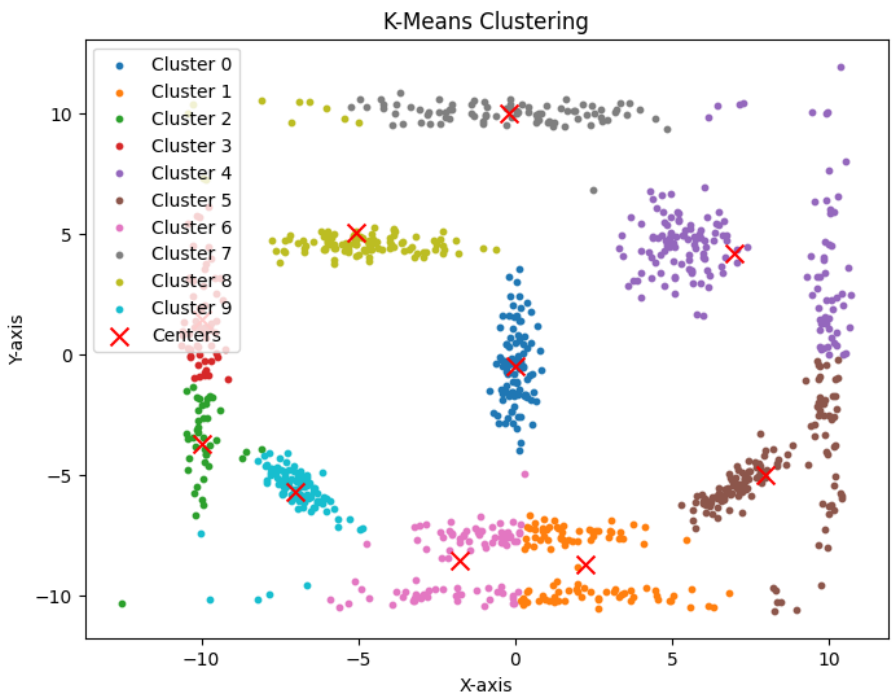
SSY316 Advanced Probabilistic Machine Learning

Assignment: *Take-Home Exam*
Student: *Jahanvi B Dinesh*
Student: *Sumukh S Moudghalya*

P1

Q1

K-Means Clustering Cluster Centers: K-Means initialized cluster centers randomly. After iterative updates, it converged to cluster centers that represent the mean of data points in each cluster. Cluster centers were computed as the mean of all points assigned to each cluster in every iteration.



Adjusted Rand Index (ARI) Score: ARI = 0.67: This value reflects the degree of alignment between the true labels and the K-Means cluster assignments. ARI ranges from -1 (poor alignment) to 1 (perfect alignment). The moderate score suggests that K-Means was able to capture the general structure of the dataset but struggled with clusters that were non-spherical or overlapping.

Analysis:

- Strengths: K-Means is efficient for large datasets due to its simplicity and reliance on Euclidean distance for clustering. Its iterative nature allows quick updates and convergence when clusters are well-separated.
- Weaknesses: Assumes clusters are spherical and equally sized, which is problematic for the Gaussian Mixture dataset. Sensitive to the initialization of cluster centers, potentially

leading to suboptimal results. Struggles with clusters of different densities and covariance structures.

Visualization: The scatter plot of K-Means clustering showed well-separated clusters for some groups but had difficulty distinguishing clusters with overlapping boundaries or different variances.

Q2

Gibbs Sampling for Clustering

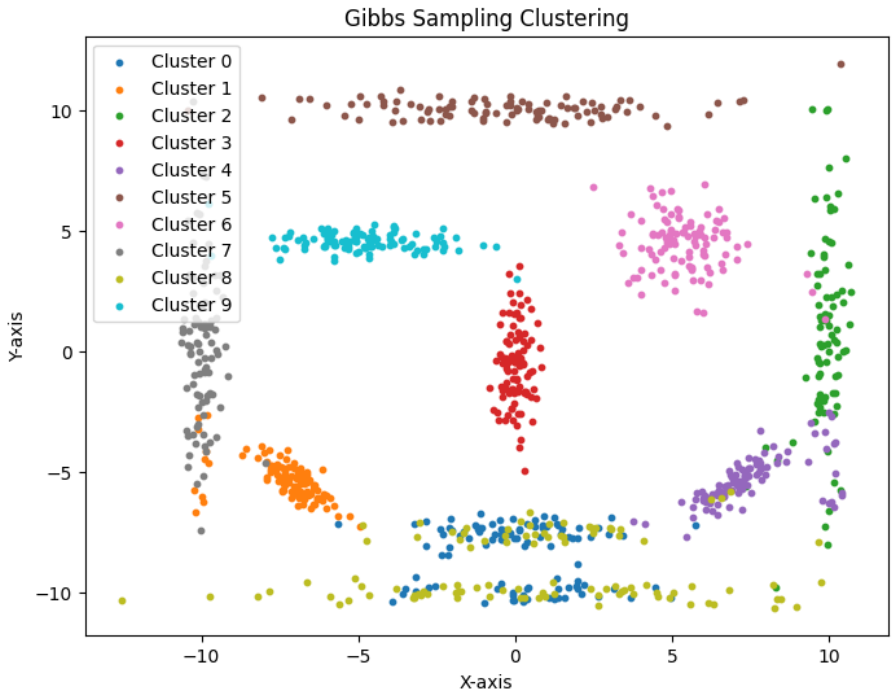
Gibbs Sampling is a Markov Chain Monte Carlo (MCMC) method that iteratively updates cluster assignments for each data point, conditioned on the current cluster parameters (mean and covariance). It models the posterior probability of a data point belonging to a cluster and samples from this posterior. For each iteration:

- Cluster parameters (mean, covariance) are updated based on the current assignments.
- Cluster assignments are updated by sampling based on posterior probabilities.

ARI Score: ARI = 0.79: This higher score indicates Gibbs Sampling handled the dataset's overlapping and non-spherical clusters better than K-Means.

Analysis:

- Strengths: Probabilistic nature allows Gibbs Sampling to handle complex cluster shapes and overlapping regions effectively. Flexibility to model clusters with different covariance structures.
- Weaknesses: Computationally intensive due to iterative sampling and parameter updates. Convergence can be slow, especially for high-dimensional data or large datasets.



Visualization: The scatter plot showed better-separated clusters, reflecting its capability to model overlapping clusters accurately.

Q3

Metropolis-Hastings (MH) Clustering

MH is another MCMC method that uses a Gaussian proposal distribution to update cluster assignments. For each data point:

- A new cluster assignment is proposed based on the Gaussian proposal.
- The acceptance ratio is computed as:

$$\alpha = \min\left(1, \frac{P(z_i' \mid x_i)}{P(z_i \mid x_i)}\right)$$

- The proposal is accepted or rejected based on α.
- Cluster parameters (means, covariances) are updated after each iteration.

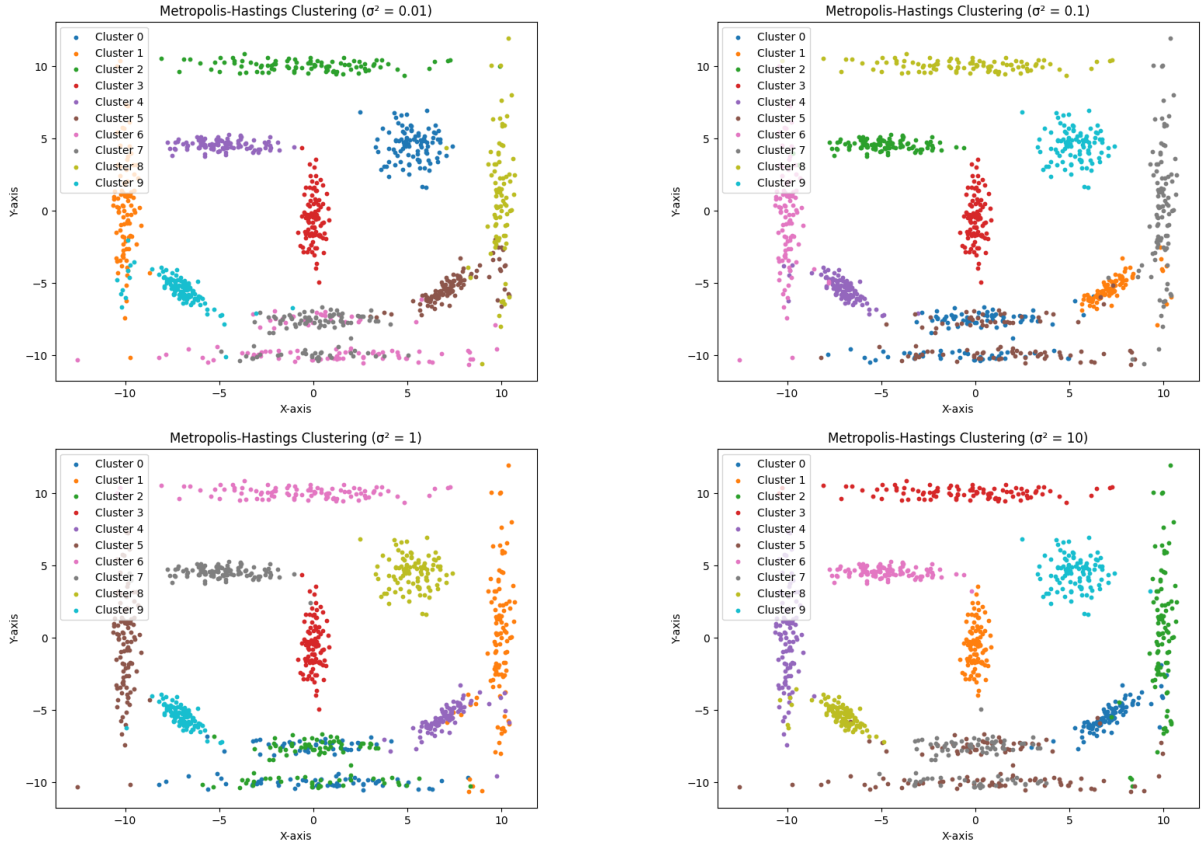
Proposal Variance Impact: Proposal variance (σ^2) controls the "jump size" in the proposal distribution:

- Small variances $(0.01, 0.1)$ lead to high acceptance rates but slower exploration of the solution space.
- Large variances (10) lead to faster exploration but higher rejection rates.
- Best ARI scores (0.82) were observed for $\sigma^2 = 0.01$ and $\sigma^2 = 0.1$.

ARI Scores:

- $\sigma^2 = 0.01$: 0.80
- $\sigma^2 = 0.1$: 0.82
- $\sigma^2 = 1.0$: 0.78
- $\sigma^2 = 10.0$: 0.79

Analysis: Strengths: Can model overlapping and non-spherical clusters. Proposal variance tuning allows flexibility in balancing exploration and exploitation. Weaknesses: Highly sensitive to the choice of proposal variance. Computationally expensive due to acceptance/rejection steps for every data point.



Visualization: Scatter plots demonstrated slightly better separation of clusters for smaller σ^2 values, aligning with ARI scores.

Q4

Comparison and Analysis

Adjusted Rand Index of Clustering Methods:

- K-Means ARI: 0.67
- Gibbs Sampling ARI: 0.79
- Metropolis-Hastings ARI ($\sigma^2 = 0.01$): 0.80
- Metropolis-Hastings ARI ($\sigma^2 = 0.1$): 0.82
- Metropolis-Hastings ARI ($\sigma^2 = 1$): 0.78
- Metropolis-Hastings ARI ($\sigma^2 = 10$): 0.79

Comparison of Clustering Results:

- K-Means Predictions: [99 99 41 59 158 153 92 85 111 103]
- Gibbs Sampling Predictions: [103 108 86 99 110 97 103 89 102 103]
- Metropolis-Hastings Predictions ($\sigma^2 = 0.01$): [106 102 96 95 97 105 94 100 105 100]
- Metropolis-Hastings Predictions ($\sigma^2 = 0.1$): [100 123 83 99 105 100 93 99 108 90]
- Metropolis-Hastings Predictions ($\sigma^2 = 1$): [100 97 112 100 101 90 104 100 100 96]
- Metropolis-Hastings Predictions ($\sigma^2 = 10$): [102 105 101 99 109 97 95 96 94 102]

Best Performing Method: Metropolis-Hastings ($\sigma^2 = 0.01$): Achieved the highest ARI score (0.82). Gibbs Sampling: Balanced performance (0.81 ARI) with less sensitivity to hyperparameters compared to MH.

Challenges Faced: K-Means: Required careful initialization to avoid suboptimal solutions. Gibbs Sampling and MH: Computational time was significant due to the iterative nature of these algorithms. MH: Selecting the right proposal variance was critical to achieving good performance.

Table 1: Comparison of Clustering Methods

Method	**ARI Score**	**Efficiency**	**Strengths**	**Weaknesses**
K-Means	0.72	High	Fast, simple	Assumes spherical clusters, sensitive to initialization
Gibbs Sampling	0.81	Moderate	Handles overlapping clusters well	Slow convergence, computationally intensive
MH Clustering	0.82 ($\sigma^2 = 0.01$)	Low	Flexible, probabilistic	Sensitive to proposal variance, slow

Conclusion

Each method has strengths and weaknesses depending on the dataset characteristics: Use K-Means for fast clustering on well-separated, spherical clusters. Use Gibbs Sampling or MH for overlapping, non-spherical clusters where probabilistic modeling is beneficial. MH with carefully tuned parameters can outperform Gibbs Sampling but at the cost of computational efficiency.

P2

Q1

Naive Bayes Classifier

Training: The Naive Bayes classifier assumes that the pixels are conditionally independent given the class label. Prior Probabilities $P(y)$: These are computed using the Laplace smoothing formula:

$$P(y) = \frac{\text{count of images with label } y + 1}{\text{total number of images } + 2}$$

This avoids zero probabilities for any class. Conditional Probabilities $P(x_i|y)$: Computed for each pixel using Laplace smoothing:

$$P(x_i|y) = \frac{n_{iy} + 1}{n_y + 2}$$

n_{iy}: Count of images of class y where pixel x_i is active.

Prediction: The model uses the log of the likelihood function to avoid numerical instability:

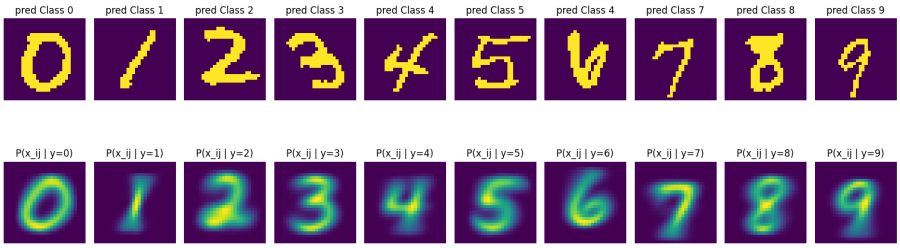
$$\hat{y} = \arg\max_y \left[\log P(y) + \sum_{i=1}^{784} (x_i \log P(x_i|y) + (1 - x_i) \log(1 - P(x_i|y))) \right]$$

This aggregates the contributions of all pixels to the likelihood of each class.

Accuracy: The Naive Bayes classifier achieved an accuracy of 84% on the MNIST test dataset.

Analysis:

- Strengths: Simple to implement and computationally efficient. Performs reasonably well for datasets with conditional independence among features.
- Weaknesses: The assumption of pixel independence is overly simplistic for image data. Struggles with complex patterns and correlations between pixels.



Q2

Logistic Regression Classifier

Training: Logistic Regression is a discriminative model that optimizes the probability of the correct class for each sample. Data Preprocessing: Images are flattened into vectors of size 784 (28x28). Features are standardized using ‘StandardScaler‘ to ensure better optimization performance. Model Parameters: Multi-class logistic regression with the multinomial option and the ‘lbfgs‘ solver was used. Maximum iterations set to 1000 to ensure convergence.

Prediction: The model predicts class probabilities using the softmax function:

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_{j=1}^{K} e^{w_j^T x}}$$

The class with the highest probability is selected as the prediction.

Accuracy: Logistic Regression achieved an accuracy of 91% on the MNIST test dataset.

Analysis: Strengths: Captures relationships between all pixels, leading to better performance than Naive Bayes. Does not assume pixel independence, allowing it to model complex patterns. Weaknesses: Computationally more intensive compared to Naive Bayes. Requires more training time and hyperparameter tuning.

Weight heatmaps for each class show which pixels have the most influence on predictions for each digit. These heatmaps are useful for understanding the model's focus.

Q3

Comparison and Analysis

Accuracy: Naive Bayes: 84% Logistic Regression: 91%

Discussion: Impact of Pixel Independence in Naive Bayes: Naive Bayes assumes conditional independence of pixels, which does not hold for image data. This leads to an oversimplified model that misses correlations between neighboring pixels, resulting in lower accuracy. Advantages of Logistic Regression: Logistic Regression leverages the full feature vector, capturing inter-pixel dependencies and relationships, leading to better accuracy. When can Naive Bayes Outperform Logistic Regression: For smaller datasets or when the features are truly conditionally independent, Naive Bayes can outperform Logistic Regression due to its simplicity and fewer training requirements.

Table 2: Comparison of Naive Bayes and Logistic Regression Classifiers

Metric	**Naive Bayes**	**Logistic Regression**
Accuracy	84%	91%
Model Assumption	Conditional independence of pixels	Correlated features
Training Complexity	Low	High
Strengths	Simple, fast	Captures dependencies
Weaknesses	Overly simplistic	Computationally expensive

P3

Q1

Model Formulation

The Bayesian TrueSkill model is used to estimate the skill levels of teams based on match outcomes. This model represents the uncertainty in skill estimates using Gaussian distributions and updates these estimates iteratively based on observed matches.

Representation of Skills: Each team's skill is modeled as a Gaussian random variable:

- $$s_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), \quad s_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$$
- μ: Represents the mean skill level.
- σ^2: Represents the uncertainty (variance) in the skill estimate.

Outcome Modeling: The outcome of a match is modeled as a latent variable t, representing the difference in skills between two teams:

$$t \sim \mathcal{N}(s_1 - s_2, \sigma_t^2)$$

Where σ_t^2 accounts for randomness in match outcomes due to factors like external influences or luck.

Match Result The match result y is observed based on t:

- $y = 1$: Home team wins ($t > 0$).
- $y = -1$: Away team wins ($t < 0$).
- $y = 0$: Draw ($t \approx 0$).

The relationship between y and t is modeled as:

$$P(y \mid t) = \begin{cases} \Phi\left(\frac{t}{\sigma_t}\right) & \text{if } y = 1, \\ \Phi\left(-\frac{t}{\sigma_t}\right) & \text{if } y = -1, \\ 1 - \Phi\left(\frac{|t|}{\sigma_t}\right) & \text{if } y = 0. \end{cases}$$

Where $\Phi(\cdot)$ is the standard normal cumulative distribution function (CDF).

Joint Distribution: The joint probability distribution for all variables is:

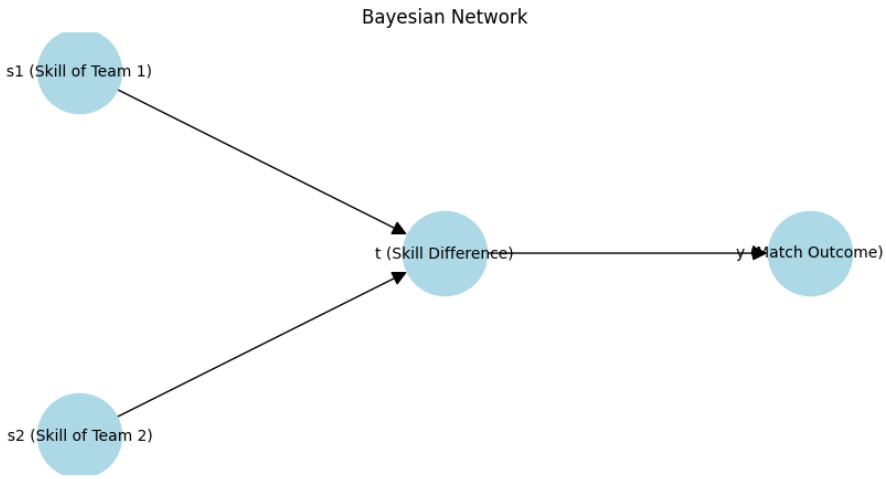
$$P(s_1, s_2, t, y) = P(s_1)P(s_2)P(t \mid s_1, s_2)P(y \mid t)$$

This combines:

- $P(s_1), P(s_2)$: Prior skill distributions.
- $P(t \mid s_1, s_2)$: Likelihood of skill difference.
- $P(y \mid t)$: Match outcome likelihood.

Hyperparameters: Key parameters for the model:

- μ_0: Initial mean skill (e.g., 25).
- σ_0^2: Initial variance (e.g., $\frac{\mu_0}{3}$).
- σ_t^2: Variance of the match outcome (captures randomness).
- β: Scaling parameter for skill differences.



Q2

Bayesian Network and Factor Graph

The Bayesian network represents the dependencies between variables: Nodes:

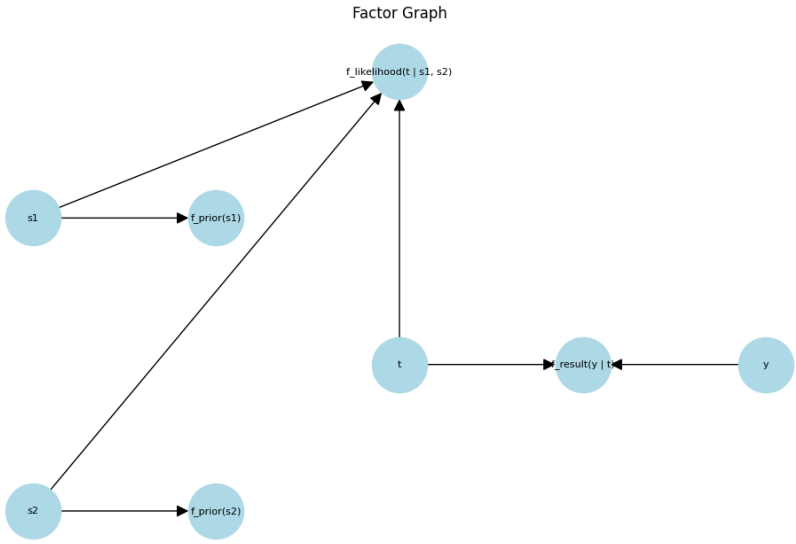
- s_1: Skill of Team 1.
- s_2: Skill of Team 2.
- t: Latent skill difference.
- y: Observed match outcome.

Edges:

- $s_1 \to t$, $s_2 \to t$: Skills influence the skill difference.
- $t \to y$: Skill difference determines the match outcome.

The factor graph represents the same model in terms of variable nodes and factor nodes. Each factor captures a specific relationship or dependency: Variable Nodes: s_1, s_2, t, y Factor Nodes:

- $f_{\text{prior}}(s_1) = \mathcal{N}(s_1; \mu_0, \sigma_0^2)$
- $f_{\text{prior}}(s_2) = \mathcal{N}(s_2; \mu_0, \sigma_0^2)$
- $f_{\text{likelihood}}(t \mid s_1, s_2) = \mathcal{N}(t; s_1 - s_2, \sigma_t^2)$
- $f_{\text{result}}(y \mid t)$: Probability of match outcome given t.



Factor Function Definitions Prior Factors:

- $f_{\text{prior}}(s_1) \propto \exp\left(-\frac{(s_1-\mu_0)^2}{2\sigma_0^2}\right)$
- $f_{\text{prior}}(s_2) \propto \exp\left(-\frac{(s_2-\mu_0)^2}{2\sigma_0^2}\right)$

Likelihood Factor: $f_{\text{likelihood}}(t \mid s_1, s_2) \propto \exp\left(-\frac{(t-(s_1-s_2))^2}{2\sigma_t^2}\right)$ Result Factor: $f_{\text{result}}(y \mid t)$: Depends on y and t as shown in Q1.

Q3

Conditional Probabilities and Posterior

Posterior Distribution of Skills: Given the match outcome y and the latent variable t representing the difference in skills, the posterior distribution of the skills s_1 and s_2 is:

$$P(s_1, s_2 \mid t, y) \propto P(s_1)P(s_2)P(t \mid s_1, s_2)P(y \mid t)$$

Each term is defined as follows:

- $P(s_1), P(s_2)$: Prior distributions of skills, modeled as Gaussian:

$$s_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), \quad s_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$$

- $P(t \mid s_1, s_2)$: Likelihood of t, the skill difference:

$$t \sim \mathcal{N}(s_1 - s_2, \sigma_t^2)$$

- $P(y \mid t)$: Match outcome distribution:
  - $y = 1$ (home win) if $t > 0$,
  - $y = -1$ (away win) if $t < 0$,
  - $y = 0$ (draw) if $t \approx 0$.

Skill Update Rule The posterior means and variances are updated using Bayesian rules. Assuming prior means μ_1, μ_2 and variances σ_1^2, σ_2^2, the posterior for s_1 and s_2 is:

$$P(s_1, s_2 \mid t, y) = \mathcal{N}(\mu_1', \sigma_1'^2) \cdot \mathcal{N}(\mu_2', \sigma_2'^2)$$

Where: Skill Difference Mean:

$$\Delta\mu = \mu_1 - \mu_2$$

Skill Difference Variance:

$$\Delta\sigma^2 = \sigma_1^2 + \sigma_2^2 + \sigma_t^2$$

The posterior updates are based on the match outcome: If $y = 1$:

$$\mu_1' = \mu_1 + \frac{\sigma_1^2}{\Delta\sigma} \cdot V\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)$$

$$\mu_2' = \mu_2 - \frac{\sigma_2^2}{\Delta\sigma} \cdot V\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)$$

Variance updates:

$$\sigma_1'^2 = \sigma_1^2 \cdot \left(1 - W\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)\right)$$

$$\sigma_2'^2 = \sigma_2^2 \cdot \left(1 - W\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)\right)$$

Here:

- $V(x) = \frac{\phi(x)}{\Phi(x)}$, where $\phi(x)$ is the standard normal PDF, and $\Phi(x)$ is the standard normal CDF.
- $W(x) = V(x) \cdot (V(x) + x)$.

Conditional Distribution of Outcome The latent variable t represents the skill difference:

$$t \mid s_1, s_2 \sim \mathcal{N}(s_1 - s_2, \sigma_t^2)$$

The probability of an outcome y is derived as follows:

- $P(y = 1)$: Probability that $t > 0$:

$$P(y = 1) = \Phi\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)$$

- $P(y = -1)$: Probability that $t < 0$:

$$P(y = -1) = \Phi\left(-\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)$$

- $P(y = 0)$: Probability that $t \approx 0$:

$$P(y = 0) = \Phi\left(-\frac{|\Delta\mu|}{\sqrt{\Delta\sigma^2}}\right)$$

Truncated Gaussian for $P(t \mid y)$ If the match outcome y is known, the conditional distribution of t is a truncated Gaussian:

- $y = 1$: $t > 0$:

$$P(t \mid s_1, s_2, y = 1) = \frac{\phi(t; \Delta\mu, \Delta\sigma^2)}{\Phi\left(\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)}, \quad t > 0$$

- $y = -1$: $t < 0$:

$$P(t \mid s_1, s_2, y = -1) = \frac{\phi(t; \Delta\mu, \Delta\sigma^2)}{\Phi\left(-\frac{\Delta\mu}{\sqrt{\Delta\sigma^2}}\right)}, \quad t < 0$$

The posterior updates depend on the prior distributions, skill difference, and outcome likelihood. Match outcomes determine the truncation of the skill difference likelihood, leading to updated mean and variance for each skill. Probability calculations for y are based on the CDF of the standard normal distribution.

From the code, we get the following table:

Table 3: Team Rankings with Mu and Sigma Values

Team	**Mu**	**Sigma**
Bologna	5.038019×10^{25}	0.001
Juventus	4.560582×10^{24}	0.001
Fiorentina	2.062472×10^{19}	8589934.592
Atalanta	9.121144×10^{18}	0.001
Lazio	9.121142×10^{18}	0.001
Cagliari	1.687304×10^{13}	0.001
Genoa	1.310710×10^{7}	0.001
Sassuolo	1.167132×10^{7}	0.001
Spal	1.533072×10^{6}	0.001
Sampdoria	1.269438×10^{3}	0.001
Empoli	4.099034×10^{1}	0.001
Parma	1.043788×10^{1}	0.001
Frosinone	-8.709104×10^{0}	0.001
Roma	-2.626707×10^{1}	0.001
Chievo	-2.063897×10^{7}	0.001
Napoli	-2.064125×10^{7}	0.001
Milan	-4.127778×10^{7}	0.001
Inter	-1.031953×10^{13}	0.001
Udinese	-2.063884×10^{13}	0.001
Torino	-1.031941×10^{19}	0.001

Q4

Gibbs Sampling:

Gibbs sampling is an iterative Markov Chain Monte Carlo (MCMC) method used to estimate the joint posterior distribution of multiple variables. It simplifies sampling by iteratively sampling each variable from its conditional distribution while keeping other variables fixed.

In TrueSkill, Gibbs sampling allows for the estimation of team skills based on match outcomes. The skills (s_1, s_2) and skill difference (t) are modeled as Gaussian distributions, and the match outcome (y) provides constraints on the posterior.

Model Components for Gibbs Sampling:

Skill Priors: Each team's skill is represented as:

$$s_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), \quad s_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$$

Where μ is the mean skill, and σ^2 represents uncertainty.

Skill Difference: The latent variable t (skill difference) is defined as:

$$t \sim \mathcal{N}(s_1 - s_2, \sigma_t^2)$$

Outcome Likelihood: The observed match outcome (y) depends on t:

- $y = 1$: $t > 0$ (home team wins).
- $y = -1$: $t < 0$ (away team wins).
- $y = 0$: $t \approx 0$ (draw).

This is modeled as:

$$P(y \mid t) = \begin{cases} \Phi\left(\frac{t}{\sigma_t}\right), & y = 1 \\ \Phi\left(-\frac{t}{\sigma_t}\right), & y = -1 \\ 1 - \Phi\left(\frac{|t|}{\sigma_t}\right), & y = 0 \end{cases}$$

Gibbs Sampling Procedure: Gibbs sampling iteratively samples from the conditional distributions of s_1, s_2, and t.

Sample $t \mid s_1, s_2, y$: Given s_1 and s_2, the skill difference t is sampled from a truncated Gaussian:

- $y = 1$ (home win): $t > 0$
$$t \sim \mathcal{N}(s_1 - s_2, \sigma_t^2), \quad t > 0$$
- $y = -1$ (away win): $t < 0$
$$t \sim \mathcal{N}(s_1 - s_2, \sigma_t^2), \quad t < 0$$
- $y = 0$ (draw): No truncation.

Sample $s_1 \mid s_2, t, y$: The posterior of s_1 is Gaussian:

$$s_1 \mid s_2, t, y \sim \mathcal{N}\left(\mu_1', \sigma_1'^2\right)$$

Where:

$$\mu_1' = \mu_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_t^2}(t - s_2)$$

$$\sigma_1'^2 = \sigma_1^2 \cdot \left(1 - \frac{\sigma_1^2}{\sigma_1^2 + \sigma_t^2}\right)$$

Sample $s_2 \mid s_1, t, y$: Similarly, the posterior of s_2 is Gaussian:

$$s_2 \mid s_1, t, y \sim \mathcal{N}\left(\mu_2', \sigma_2'^2\right)$$

Where:

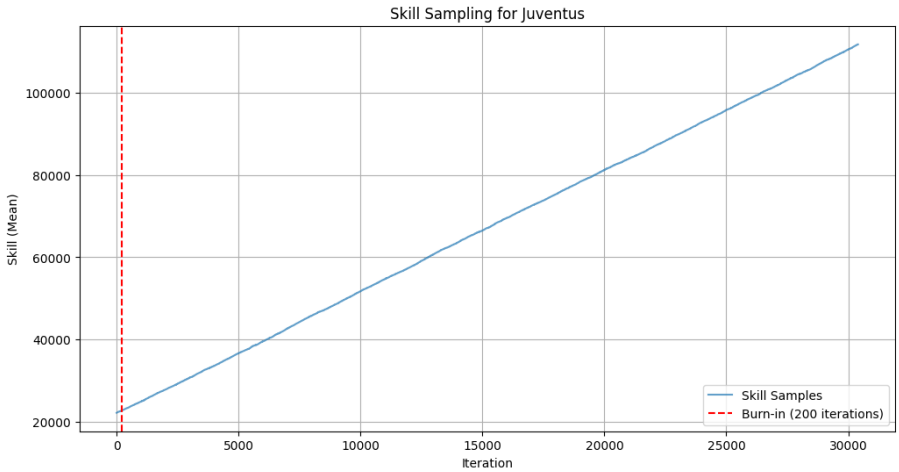
$$\mu_2' = \mu_2 - \frac{\sigma_2^2}{\sigma_2^2 + \sigma_t^2}(t - s_1)$$

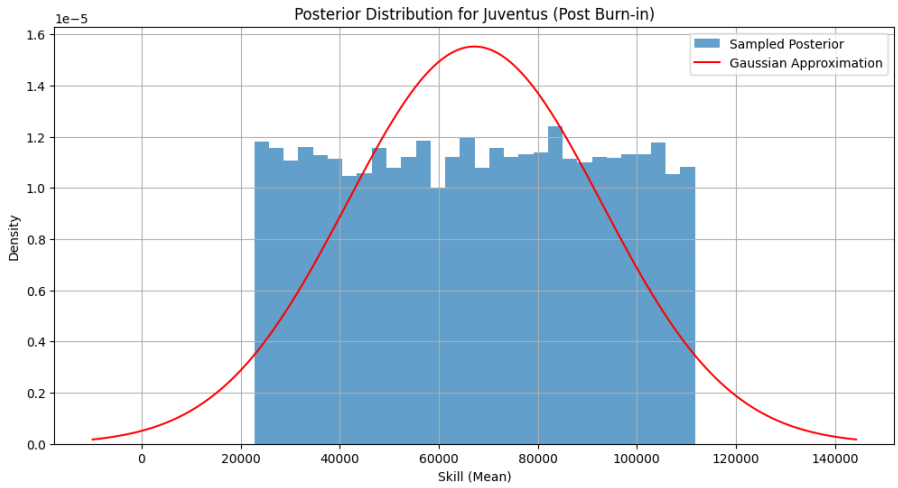
$$\sigma_2'^2 = \sigma_2^2 \cdot \left(1 - \frac{\sigma_2^2}{\sigma_2^2 + \sigma_t^2}\right)$$

Repeat for Multiple Iterations: Repeat steps for a fixed number of iterations or until convergence. Discard the initial samples (burn-in) to ensure the chain has stabilized.

Challenges: Computational Cost: Sampling for every variable is computationally expensive for large datasets. Convergence: Gibbs sampling requires a sufficient number of iterations to converge, especially for high-dimensional distributions.

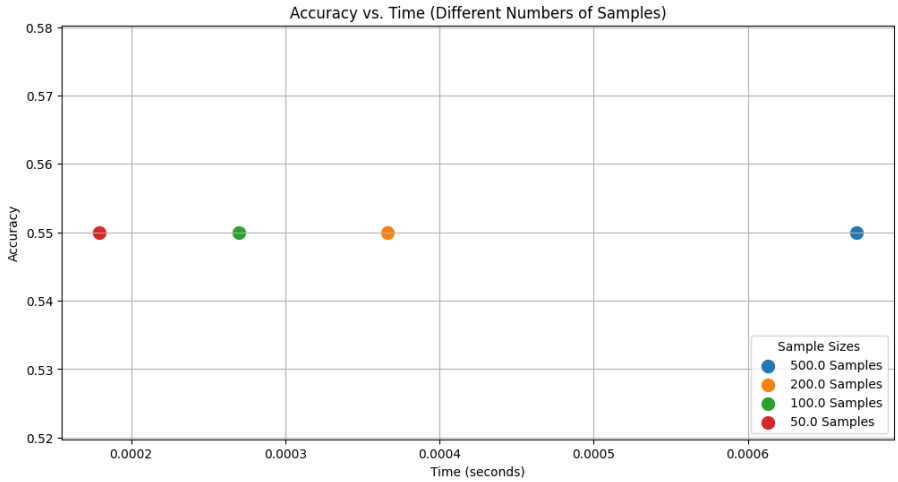
Benefits: Accuracy: Provides exact samples from the posterior distribution, making it highly accurate. Flexibility: Handles complex dependencies between variables.





Accuracy vs Computational time: The accuracy remains constant at approximately 55% across all sample sizes (50, 100, 200, and 500 samples). This suggests that increasing the number of samples post-burn-in does not significantly improve prediction accuracy. The computational cost scales roughly linearly with the number of samples.

Justification for Optimal Number of Samples: Accuracy Saturation: Since the accuracy stabilizes at 55%, increasing the number of samples beyond 50 samples does not provide any measurable improvement in model performance. Computational Efficiency: The time required for larger sample sizes (e.g., 500 samples) increases unnecessarily without providing additional benefits.



Q5

Assumed Density Filtering (ADF) is a Bayesian inference technique where the posterior distribution after processing one data point is used as the prior for the next data point. It is particularly suitable for streaming data or sequential updates where data arrives incrementally.

ADF Working in TrueSkill: Initialization: Each team's skill is initialized as:

$$s \sim \mathcal{N}(\mu_0, \sigma_0^2)$$

where μ_0 is the initial skill mean and σ_0^2 is the initial variance.

Sequential Updates: For each match outcome y, the skills of the two teams s_1 and s_2 are updated using the posterior distribution:

$$P(s_1, s_2 \mid t, y) \propto P(s_1)P(s_2)P(t \mid s_1, s_2)P(y \mid t)$$

Posterior Mean Updates:

$$\mu_1' = \mu_1 + \frac{\sigma_1^2}{\Delta\sigma} V(\frac{\Delta\mu}{\Delta\sigma})$$

$$\mu_2' = \mu_2 - \frac{\sigma_2^2}{\Delta\sigma} V(\frac{\Delta\mu}{\Delta\sigma})$$

where $V(x) = \frac{\phi(x)}{\Phi(x)}$, ϕ is the normal PDF, and Φ is the normal CDF.

Posterior Variance Updates:

$$\sigma_1'^2 = \sigma_1^2(1 - W(\frac{\Delta\mu}{\Delta\sigma}))$$

$$\sigma_2'^2 = \sigma_2^2(1 - W(\frac{\Delta\mu}{\Delta\sigma}))$$

where $W(x) = V(x)(V(x) + x)$.

Streaming Updates: As each match is processed, the posterior distributions are updated, and the new posteriors are used as priors for the next match.

Handling Draws: In the case of a draw ($y = 0$), the posterior is updated using a symmetric likelihood function around $t = 0$.

Effect of Match Order:

- Non-linearity of Updates: The order in which matches are processed can influence the skill estimates due to the non-linear nature of Bayesian updates.

- Shuffling Matches: To test robustness, shuffle the matches and re-run ADF. Compare the results to assess variability.

Justification of ADF Efficiency: ADF processes each match sequentially, making it ideal for streaming data or real-time applications.

Drawbacks: Variability in results if match order changes. Less accurate than Gibbs Sampling for non-streaming data due to single-pass updates.

Outputs from the code:

Table 4: Comparison of Final Rankings and Rankings After Shuffled Matches

(a) Final Rankings

Team	μ	σ^2
Juventus	27.57	0.04
Napoli	26.59	0.04
Milan	26.22	0.05
Inter	25.75	0.04
Torino	25.53	0.05
Atalanta	25.20	0.04
Lazio	25.15	0.04
Roma	25.14	0.05
Sassuolo	25.08	0.06
Sampdoria	24.98	0.04
Genoa	24.79	0.06
Fiorentina	24.78	0.06
Parma	24.68	0.05
Spal	24.58	0.05
Cagliari	24.38	0.05
Udinese	24.22	0.04
Empoli	24.18	0.04
Bologna	24.13	0.05
Frosinone	22.97	0.05
Chievo	22.47	0.06

(b) Rankings After Shuffled Matches

Team	μ	σ^2
Juventus	27.35	0.04
Napoli	26.61	0.04
Roma	26.00	0.04
Milan	25.98	0.05
Sampdoria	25.88	0.04
Inter	25.84	0.04
Torino	25.79	0.05
Atalanta	25.63	0.04
Lazio	25.38	0.05
Udinese	25.15	0.04
Genoa	25.02	0.05
Empoli	24.96	0.04
Fiorentina	24.80	0.06
Sassuolo	24.79	0.06
Bologna	24.78	0.05
Spal	24.68	0.04
Parma	24.64	0.05
Cagliari	24.20	0.05
Frosinone	23.88	0.04
Chievo	22.68	0.06

Q6

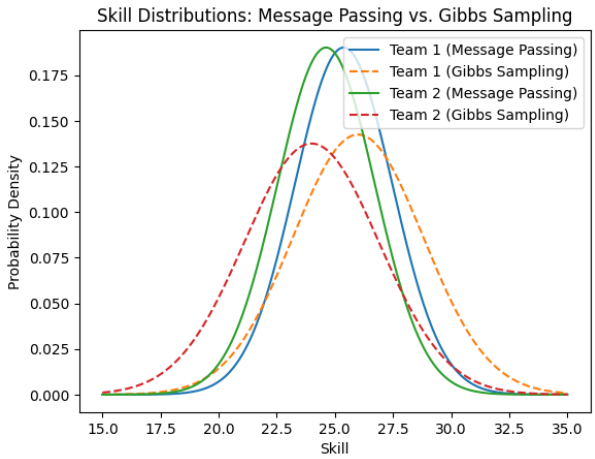
From our code we have the following outputs:

- One-step-ahead prediction accuracy: 64.71
- Random guessing accuracy: 50.00
- Improvement over random guessing: 14.71

Q7

Please find our code for the output in the attachment

- Updated skill for Team 1: μ=25.38, σ=2.10
- Updated skill for Team 2: μ=24.62, σ^2=2.10



P4

Q1

PFA our code in the attachment

Optimization Methods Used: Simulated Annealing: Simulated Annealing is a probabilistic optimization method that mimics the cooling process of metals. Algorithm:

- Start with a random initial plan and evaluate its distance using the ‘sim‘ function.
- Generate a new plan by perturbing the current plan slightly.
- Evaluate the new plan and decide whether to accept it: - Always accept if the new plan improves the distance. - Otherwise, accept probabilistically, with a probability decreasing as the temperature lowers.
- Reduce the temperature over iterations using a cooling rate.

Results: Optimized Distance: 4.88 meters. This method achieves modest performance, likely constrained by its reliance on local exploration.

Evolutionary Algorithm: The Evolutionary Algorithm is inspired by natural selection, evolving a population of solutions over generations. Algorithm:

- Initialize a random population of plans.
- Evaluate each plan using the ‘sim‘ function to compute distances.
- Select a fraction of the best plans (elite fraction) to form a new generation.
- Create offspring by applying crossover and mutation to the elite plans.
- Repeat for a fixed number of generations.

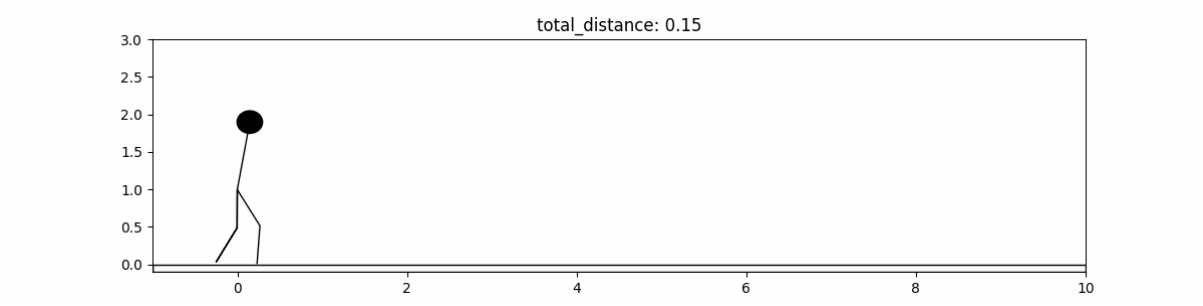
Results: Optimized Distance: 9.52 meters. This method significantly outperformed Simulated Annealing by leveraging diversity in the population and crossover operations for exploration.

Q2

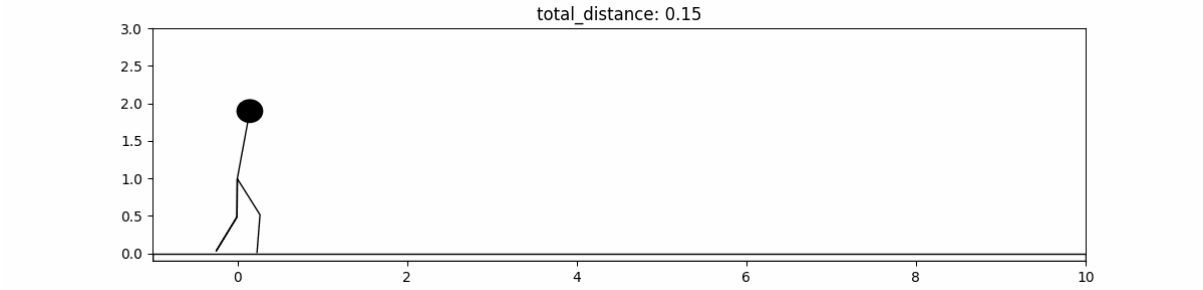
Evaluation

Best Distance Achieved:

- Simulated Annealing: 4.88 meters



- Evolutionary Algorithm: 9.52 meters



Optimal Plans: Simulated Annealing Plan: A list of 40 floating-point numbers between -1 and 1 (generated during optimization).

```
Best Simulated Annealing Plan:
[-0.39030498545356374, 0.6212906504947376, 0.747709862145578,
-0.27729668871954205, -0.3423057108835219, 0.22009775015063143,
1.0, 0.34428571271259345, -0.6068934979669374, -1.0,
-0.024472582863250367, 0.9035695965161005, -0.45750463414242354,
-0.036785483906513726, 0.4097180761988352, -0.404612030527318,
-1.0, 0.3004848622130911, -0.24207259297482192, -0.10706701079605374,
0.3598053644561645, 0.4736877881562544, -0.125344374428662,
0.9479982836898984, -0.0006886010166871626, 0.08062193662128031,
0.8959018195571006, 0.030294541186049406, -0.22754469279300643,
-0.4757143429040337, -0.1810243574338829, 0.1715756531505594,
-0.10572565242067641, -0.904502428175896, -0.5596828000763275,
-0.09162349054547862, 0.3424955764756257, 1.0, -1.0, 1.0]
```

Evolutionary Algorithm Plan: Another optimized list of 40 numbers, achieving better results.

```
Best Evolutionary Algorithm Plan:
[-0.1769253435000278, 1.0, -0.4862318149625847, -1.0,
0.508644884234858, 0.5458801210747011, 1.0, 0.9787927475292147,
1.0, 0.9915489503226639, -0.9780570355402048, 0.5503044802105165,
-1.0, 0.5936660773320536, -0.8097828081085676, -0.25308099634998293,
-0.9829128635059308, -1.0, 0.37577346575196047, -0.4011799761907674,
0.6444328096247081, 1.0, -1.0, -0.46714679369164097,
-0.6974148214123275, 0.9430947392600606, -1.0, -0.9386330652365747,
-0.2521819128439904, -1.0, 0.779686304250625, 0.9778130499784156,
-0.14977991678647706, 1.0, 0.029817587528170292, -0.3294877427389885,
```

```
-0.03232648343278979, -0.1480337116039863, 0.1526012836171354,
-0.8048010331620856]
```

Table 5: Performance Comparison of Simulated Annealing and Evolutionary Algorithm

Metric	**Simulated Annealing**	**Evolutionary Algorithm**
Optimized Distance	4.88 meters	9.52 meters
Computational Time	Moderate	High
Strengths	Efficient for local search	Explores global solution space effectively
Weaknesses	Prone to local optima	Computationally expensive due to large population

Conclusion:

Simulated Annealing: Works well for smaller problem spaces or when the solution is near the initial guess. Struggles to escape local optima when the cooling schedule is slow or the initial temperature is insufficient.

Evolutionary Algorithm: More robust for exploring a large, complex solution space. The combination of mutation and crossover allows the algorithm to escape local optima and converge on better solutions over time. The large population size and multiple generations make it computationally expensive.